PDF Search API — Technical Decisions & Interview Prep

Embedding model: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2

What it is: a distilled multilingual sentence-transformer (12 layers, MiniLM architecture), 384-dim output, ~470MB, trained for semantic similarity/paraphrase detection across 50+ languages.

Why this one:

- Documents are French → needs multilingual support (a plain English model like all-MiniLM-L6-v2 would perform poorly).
- Runs on CPU in seconds — no GPU requirement, satisfies the "local, no paid API" constraint.
- It's the example suggested in the test brief itself, so it's a defensible, low-risk choice.
- Small enough (~470MB) to download and load quickly for a take-home exercise.

Trade-off to flag: a larger multilingual model (e.g. intfloat/multilingual-e5-base) would likely give better retrieval quality at the cost of slower CPU inference and a bigger download.

Chunk size: 800 characters, 150 overlap (~19%)

Why fixed-size character chunking (not sentence/token-aware): simplest deterministic approach, no extra NLP dependency, language-agnostic — a pragmatic choice given the "don't over-engineer" framing of the brief.

Why 800 chars: roughly 120-150 words, safely under the ~256 token context window of the model (nothing gets silently truncated) while staying topically focused for retrieval.

Why overlap: prevents losing context for sentences/ideas that straddle a chunk boundary.

Honest weakness: still character-based, so it can cut mid-sentence or mid-table-row.

Vector store: FAISS IndexFlatIP

Why FAISS: free, local, no server/managed service required, industry-standard, also suggested in the brief.

Why Flat (exact) instead of approximate (IVF/HNSW): at the scale of a handful of PDFs, exact search is already sub-millisecond. Approximate indexing only pays off at much larger scale (millions of vectors) and would add tuning complexity with zero benefit here.

Why inner product (IP) and not L2 distance: embeddings are L2-normalized at encode time, so inner product becomes mathematically equivalent to cosine similarity — the standard metric for sentence embeddings.

PDF text extraction: pdfplumber

Why: good real-world layout handling, clean per-page access (needed for the page_number metadata field), permissive license, no compiled-binary licensing issues (unlike PyMuPDF's AGPL).

Why not OCR: the brief explicitly says not to worry about extraction quality; OCR (Tesseract) would mean bundling system binaries for a problem (scanned PDFs) that's out of scope.

Architecture choices worth explaining

- **Single Dockerfile/image** for both the ingestion CLI and the API — same codebase and dependencies; the run command selects which mode runs.
- **data/ folder as the persistence boundary** between ingestion and API — plain files on a shared volume, no database server, satisfies "no managed cloud service."
- **Metadata as a JSON list**, positionally aligned with FAISS row IDs — simplest design at this scale; doesn't support efficient single-record updates/deletes like a real DB would.
- **Index loads once at API startup**, not per request — startup cost (model + index load) is too high to pay per request. Trade-off: requires a restart after re-ingestion.
- **chunk_index resets per document** — (document_name, chunk_index) is a natural composite key, simpler than global UUIDs.
- **No LLM answer generation** — explicitly out of scope per the brief.

Likely tester questions — quick answers

Question	Answer
Why not a bigger/better embedding model?	CPU speed and download size vs. accuracy trade-off; this one is "good enough" and explicitly suggested.
What if top_k is huge or query is empty?	Pydantic validates top_k in [1,50] and query non-empty at the schema level (422 on violation).
How would you handle scanned/image PDFs?	Add an OCR step (Tesseract/EasyOCR) before chunking; flagged as a known limitation.
How do you evaluate search quality without labels?	Acknowledged limitation — would need a small labeled query/relevant-chunk set to compute recall@k; out of scope for the time budget.
Why FAISS over a "real" vector DB (Pinecone/Qdrant/Weaviate)?	No managed service allowed, and overkill for this data volume; FAISS is the lightest local option.

How does this scale to 10,000 PDFs?	Flat index search degrades linearly with vector count; would switch to IVF/HNSW and move metadata to SQLite/Postgres.
Why fixed full re-ingestion instead of incremental updates?	Time-boxed scope; full rebuild is simplest correct behavior. Incremental would need per-document add/remove in FAISS + metadata, with care around ID stability.
Why no reranking or hybrid (BM25 + embeddings) search?	Out of scope per brief ("not interested in advanced retrieval techniques"); would be the first quality improvement added.
Why does /search return 503 sometimes?	If ingestion hasn't run yet, the API has no index loaded — deliberate, explicit failure mode instead of silently returning nothing.
Why not use LangChain/LlamaIndex?	Wanted explicit control over chunking/embedding/storage rather than abstracted framework behavior — easier to explain every decision in review.
What does the score mean?	Cosine similarity in [-1,1] (usually 0.2-0.9 for relevant matches); higher = more semantically similar.
Biggest assumption you'd flag?	Filenames are unique, PDFs fit in a flat folder, and the corpus stays small enough for exact search to remain fast.